



SC4020 Data Analytics & Mining Project 1

**What Method of Aggregating Information from
Neighbours is the Most Effective in Link Prediction when
Various Relation Types Exist?**

Group 4

Team Member	Matriculation Number
Lee Choonggi	U2320667H
Sih Jia Qi	U2321900L
Ponyuenyong Kritchanat	U2322517E
Yalamanchili Sanjana	U2323273H

Content

Content.....	2
Abstract.....	4
1. Introduction.....	5
1.1 LightGCN.....	5
1.1.1 LightGCN Introduction.....	5
1.1.2 LightGCN Architecture.....	6
1.2 GAT.....	7
1.2.1 GAT Introduction.....	7
1.2.2 GAT Architecture.....	8
1.3 GIN.....	9
1.3.1 GIN Introduction.....	9
1.3.2 GIN Architecture.....	10
2. Methods.....	11
2.1 Datasets.....	11
2.1.1 WN18RR Dataset.....	11
2.1.2 FB15k-237 Dataset.....	12
2.1.3 CORA Dataset.....	12
2.2 Experimental Settings.....	12
2.2 R-LightGCN.....	13
2.2.1 Architecture and Relational Encoding.....	13
2.2.2 Model Structure and Layer Combination.....	14
2.2.3 Data Preprocessing and Decoding for Link Prediction.....	15
2.3 Relational Graph Attention Network (R-GAT).....	15
2.3.1 Architecture and Relational Attention.....	15
2.3.2 Propagation and Multi-Head Aggregation.....	16
2.3.3 Application and Decoding.....	16
2.4 Relational Graph Isomorphism Network (R-GIN).....	18
2.4.1 Architecture and Message Passing.....	18
2.4.2 Relational Extensions.....	19
2.4.3 Layer Combination and Embedding Aggregation.....	19
2.4.4 Decoding for Link Prediction.....	20
3. Experiments.....	21
3.1 Ablation Study on R-LightGCN.....	21
3.1.1 Decoder Ablation: DistMult vs. Dot.....	21
3.1.2 Transformation Strategy: Scalar vs. Diagonal.....	22
3.1.3 Normalization Scope: Global vs. Per-Relation.....	22
3.1.4 Results and Discussion.....	23

3.2 Ablation Study on R-GAT.....	27
3.2.1 Decoder Ablation: DistMult vs. DotProduct.....	28
3.2.2 Encoder Ablation: Attention per Relation vs. Embed Relation.....	28
3.2.3 Results and Discussion.....	29
3.3 Ablation Study on R-GIN.....	33
3.3.1 Decoder Ablation: DistMult vs. Dot.....	34
Dot Product Decoder (Relation-Independent).....	34
DistMult Decoder (Relation-Aware).....	34
Interpretation.....	35
3.3.2 Encoder Ablation: Relation Encoding Strategies.....	35
3.3.2.1 Base GIN (Homogeneous Baseline).....	35
3.3.2.3 R-GIN-FiLM (Feature-wise Linear Modulation).....	36
3.3.2.4 R-GIN-LoRA (Low-Rank Adaptation).....	37
(a) LoRA-Per-Relation.....	37
(b) Unified LoRA.....	37
3.3.3 Results and Discussion.....	37
3.4 Unified R-GAT+LoRA.....	42
3.4.2 Layer formulation.....	42
3.4.3 Encoder–decoder scoring.....	43
3.4.4 Learning objective and negative sampling.....	44
3.4.5 Computational considerations.....	44
3.4.6 Implementation flow.....	44
3.4.7 Results.....	44
3.4.8 Results Analysis.....	45
4. Conclusion.....	46
5. References.....	47
Appendix.....	49
A Performance Metrics.....	49

Abstract

This report investigates the performance of three different link prediction models. Link prediction is a fundamental task in graph analysis, essential for applications ranging from knowledge graph completion to recommender systems. The models were trained on the training data, fine-tuned using validation data, and assessed on the test data to compare the performance between the three methods. Our findings highlight the strengths and limitations of the models, providing insights into their effectiveness in similarity search tasks. We investigate the performance trade-offs among LightGCN, GAT, GIN and their relational variants (e.g., R-LightGCN, R-GAT and R-GIN adaptations designed to handle diverse edge types). Experiments are conducted on three public benchmark datasets representing various real-world graph structures and link prediction scenarios. This study offers valuable insights into the necessary structural components for effective relational graph representation learning.

Our implementations are available at: <https://github.com/BlurSotong1/sc4020-Project-1>.

1. Introduction

Graph Neural Networks (GNNs) have become an important framework for learning representations from relational data. A central operation in these models is the aggregation of information from neighbouring nodes, which determines how structural and semantic context is encoded into node embeddings. Different aggregation approaches such as mean or sum pooling, attention weighting, and relation specific propagation have been proposed, each offering a different balance between expressiveness, interpretability, and computational cost.

In the task of link prediction, where the goal is to infer missing or potential edges in a graph, the choice of aggregation strategy has a direct influence on model performance. This study examines the effectiveness of several neighbour aggregation methods, including Light Graph Convolution Network (LightGCN), Graph Attention Network (GAT), Graph Isomorphism Network (GIN), and their relational variants, to identify which approach best captures the relational patterns required for accurate link prediction.

1.1 LightGCN

1.1.1 LightGCN Introduction

LightGCN is a simple graph neural network that uses a normalized sum to aggregate data from a node's neighbors, with each neighbor sharing an equal amount of the update (He et al., 2020). The weighted total of the embeddings learnt at each layer serves as the final embedding, which LightGCN learns by linearly propagating user and item embeddings on the user-item interaction graph.

It is "light" because it eliminates feature transformations and activation functions, leaving only the necessary aspect of graph propagation. As a result, the model retains high-order connectivity while becoming quicker, simpler to train, and less likely to overfit. Each node is represented by an embedding vector $\vec{e}_i$, and information is aggregated from its neighbours through a normalized weighted sum:

$$e_i^{(k+1)} = \sum_{j \in N} \frac{1}{\sqrt{|\mathcal{N}(i)| |\mathcal{N}(j)|}} e_j^k$$

Here, $\mathcal{N}(i)$ denotes the neighbours of node i . After k layers, the final embedding is obtained by averaging the representations from all layers.

1.1.2 LightGCN Architecture

As seen in Figure 1, LightGCN spreads embeddings across the bipartite structure. The embeddings of the objects linked to each user are aggregated to update the user's embedding, and vice versa:

$$e_i^{(k+1)} = \frac{1}{K+1} \sum_{j \in N_i} e_{ij}^{(k)}$$

where $e_i^{(k+1)}$ represents the embedding (vector) of node i at the $(k+1)$ -th layer, N_i stands for the neighborhood of node i , $e_{ij}^{(k)}$ is the message or transformed feature passed from neighbor j to node i at the k -th layer.

A mathematical representation of the normalized adjacency matrix A in Figure 1 instructs the model on how to distribute data throughout the graph. LightGCN captures higher-order connections (e.g., user $\rightarrow$ item $\rightarrow$ other users $\rightarrow$ other items) by stacking numerous layers.

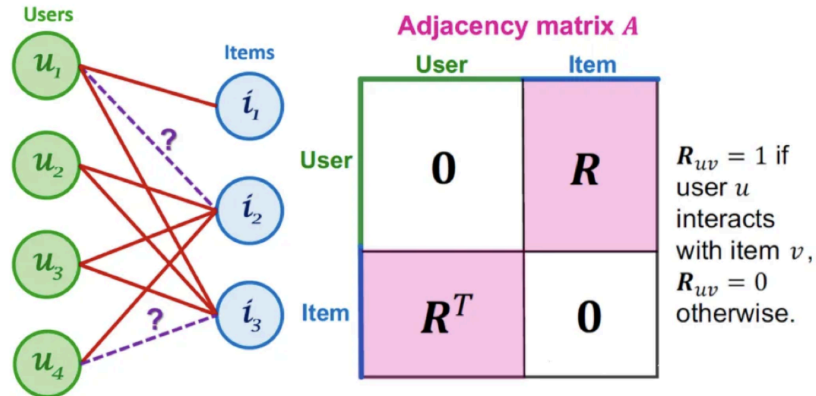


Figure 1. Node Edges between Users and Interacted Items

For link prediction, the score of a pair (u, v) is computed as the dot product of their embeddings:

$$s(u, v) = e_u^T e_v$$

where e_u and e_v respectively denote the embedding vector of node u (e.g., a user) and embedding vector of a node v (e.g., an item). If e_u and e_v point in the similar directions in embedding space, $s(u, v)$ will be large, indicating a strong likelihood of a link.

This lightweight and efficient method only considers the structure of the graph, which is frequently more significant than complex feature transformations. When it comes to recommendation tasks, LightGCN has outperformed heavier GCNs in previous studies. An average relative improvement of 16.0% over the state-of-the-art GCN-based recommender model Neural Graph Collaborative Filtering (NGCF) (Zhao et al., 2020).

1.2 GAT

1.2.1 GAT Introduction

GAT aggregates information from a node's neighbours using a learned attention mechanism, which assigns different importance weights to each neighbour during message passing. This allows the model to focus more on relevant or informative neighbours when updating node representations.

Each node i computes attention coefficients with its neighbours j as follows:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(a^T [W e_i \parallel W e_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(a^T [W e_i \parallel W e_k]))}$$

Here, the α_{ij} represents the normalised attention weight between node i and its neighbour j , W is a shared linear transformation, and a is a learnable vector defining how attention is

computed. The updated embedding of node i is obtained by aggregating the weighted messages from its neighbours:

$$e_i^{(k+1)} = \sigma \left(\sum_{j \in \mathcal{N}} \alpha_{ij} W e_j^{(k)} \right)$$

This mechanism allows GAT to adaptively learn which neighbours are most important, improving performance on tasks where relationships between nodes vary in significance. It is particularly effective for heterogeneous or noisy graphs, where uniform neighbour weighting may be suboptimal.

1.2.2 GAT Architecture

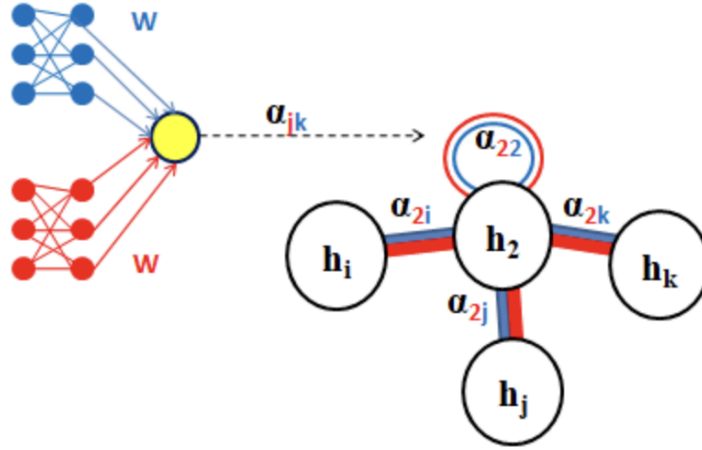


Figure 2: GAT Architecture with Attention-based Weighting

The architecture of GAT follows the general message passing framework of graph neural networks but enhances it through the use of attention-based weighting.

The computation consists of four main steps. First, each node feature vector h_i is transformed using a shared weight matrix W to produce a new feature z_i . Second, for every pair of connected nodes (i, j) , the model calculates an attention score e_{ij} based on the concatenation of their transformed features. Third, these scores are normalized with a softmax function to

obtain attention coefficients α_{ij} that represent the relative contribution of each neighbor. Fourth, the node representation is updated by aggregating the weighted sum of its neighbor's features and a nonlinear activation function.

This process enables GAT to selectively emphasize more informative neighbors and reduce the influence of less relevant ones. As shown in Figure 2, each edge between a node and its neighbor carries a learnable weight α_{ij} which reflects the importance of each connection. Figure 3 illustrates the next stage on how these raw attention scores e_{ij} are passed through the softmax layer to generate normalized weights.

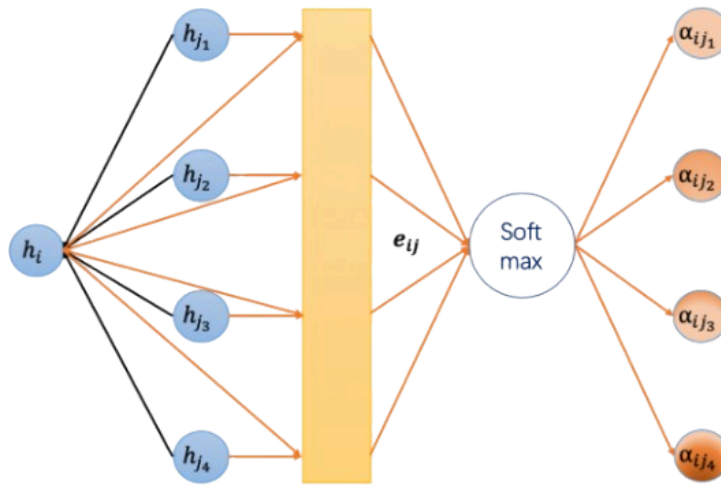


Figure 3: Normalized Weights through Softmax Layer

1.3 GIN

1.3.1 GIN Introduction

GIN aggregates information from a node's neighbours through a sum-based aggregation function, which treats each neighbour equally but enhances expressiveness by applying a multi-layer perceptron (MLP) after aggregation. This design allows GIN to learn complex and non-linear transformations of neighbourhood information, making it as powerful as the Weisfeiler-Lehman graph isomorphism test in distinguishing graph structures. At each layer, the representation of node i is updated as follows:

$$h_i^{(k+1)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)}) h_i^{(k)} + \sum_{j \in \mathcal{N}(i)} h_j^{(k)} \right)$$

Here, h_i^k represents the embedding of node i at layer k , and ϵ^k is a learnable (or fixed) scalar that controls the importance of the node's own features relative to its neighbours. The MLP captures non-linear interactions between the aggregated messages, enabling the model to learn richer representations than linear propagation methods.

By using sum aggregation (instead of mean or attention), GIN preserves information about the size and composition of a node's neighbourhood, leading to strong discriminative ability. It has shown state-of-the-art performance in tasks such as graph classification and link prediction, especially where the structure and node identity play a crucial role.

1.3.2 GIN Architecture

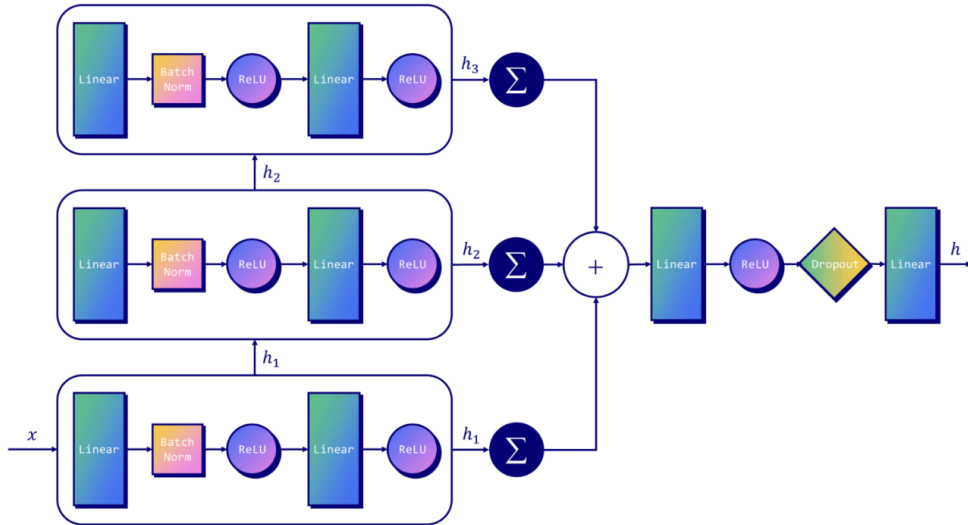


Figure 4. Overview of GIN Architecture

The Graph Isomorphism Network (GIN) is designed to maximize representational power by mimicking the expressiveness of the Weisfeiler–Lehman (WL) graph isomorphism test. Unlike GAT, which uses attention to assign different weights to neighbours, GIN aggregates all neighbour features equally and uses multi-layer perceptrons (MLPs) to transform them.

The process follows three main steps. First, each node aggregates the sum of its own feature and its neighbours' features, ensuring permutation invariance. Second, the summed features are passed through an MLP composed of linear layers, batch normalization, and ReLU activations to produce a new node embedding. Third, outputs from multiple layers (h_1, h_2, h_3) are combined through skip connections and processed by additional linear and dropout layers to form the final representation.

This architecture enables GIN to capture fine-grained structural differences between nodes that simpler aggregation schemes (like mean or max pooling) might overlook. As shown in Figure 4, the use of multiple MLP layers and residual summations allows GIN to learn powerful, discriminative graph representations while maintaining stability across different graph sizes and densities.

2. Methods

2.1 Datasets

Datasets. In our experiment, three benchmark datasets are utilized to compare the performances of GNN and relational GNN models i.e., WN18RR, FB15k-237 and CORA, each offering distinct structural and relational characteristics. Table 1 describes the statistics of the datasets and their respective splitting ratio.

Table 1. Dataset Statistics

Datasets	WN18RR	FB15K-237	CORA
Entities	40,943	14,541	2,708
Relations (typed)	11	237	2
Training Triples	86,835	272,115	8,448
Validation Triples	3,034	17,535	1,056
Test Triples	3,134	20,466	1,056

2.1.1 WN18RR Dataset

Dettmers et al. (2017) developed WN18RR, an improved version of the original WN18 dataset (Bordes et al., 2013), to solve test leaking concerns through inverse relations. Many connection pairs, including hyponyms and hypernyms, were present in the original WN18, a subset of WordNet, which enabled models to infer test triples by simply flipping training triples. In order to get around this, WN18RR eliminates these duplicate and inverted relations, leaving 93,003 triples, 40,943 entities, and 11 relations. It guarantees a more dependable and difficult standard for link prediction by eliminating inverse relations, highlighting relational patterns such as composition, symmetry, and antisymmetry.

2.1.2 FB15k-237 Dataset

A refined subset of the original Freebase FB15K dataset, FB15K-237 minimizes information leakage and redundancy (Bordes et al., 2013). The 401 most often used relations from FB15K were chosen, and inverse and near duplicate relations were then eliminated to form the dataset. Relations were deemed inverse if their entity pairs were the opposite of one another, and near duplicates if their entity pairs overlapped by at least 97%. 237 distinct relations were

left behind after this filtering procedure, producing a more accurate baseline for link prediction.

2.1.3 CORA Dataset

The CORA dataset is a citation network that was first used for machine learning and document categorization research by McCallum et al. (2000). It is made up of 5,429 citation linkages (edges) connecting 2,708 scientific papers (nodes) categorized into seven groups. Every publication is represented by a binary word vector with 1,433 dimensions that shows whether or not a certain term appears in the paper. Because of its well-defined structure and modest size, which allow for a fair evaluation of graph neural network models, the dataset is frequently used as a benchmark for graph learning tasks including node classification and link prediction.

2.2 Experimental Settings

Metrics. For each test triple (h, r, t) , in tail prediction, the model replaces the true tail t with every other entity t' and ranks all score $s(h, r, t')$. In head prediction, the head h is replaced with every other entity h' and all scores $s(h', r, t)$ are ranked. The area under curve (AUC) and ranking based metrics i.e., HITS@K where $K \in \{1, 5, 10\}$ are adopted. For both types of metrics, a higher value represents better model performance. Refer to Appendix A for the formal definition of each evaluation metric used.

Hyperparameter Ranges. All models were optimized using the Adam optimizer with a fixed learning rate of $\lambda = 1e-3$. The embedding dimensions were chosen within $\{64, 128\}$, and the number of propagation layers was varied from $\{3, 4\}$ to study the impact of message-passing depth. Weight decay was tuned within $\{0, 1e-4\}$, and weight decay within $\{0, 1e-4\}$.

Evaluation Settings. Relationships that were inverses of another (i.e., their entity pairs were in reverse order) or overlapped by at least 97% were excluded. A more realistic benchmark for link prediction was produced by this technique, leaving 237 unique relations.

The batch size was 2048. The predetermined patience was 10. Early stopping was added during model training to avoid overfitting and needless computation if the validation AUC did not improve for a certain period of epochs determined by the patience parameter. The final testing was carried out using the model checkpoint from the epoch with the greatest validation AUC.

2.2 R-LightGCN

The effective LightGCN model for relational graph learning has been modified to create the Relational Light Graph Convolutional Network (R-LightGCN), which is intended for applications such as Knowledge Graph Completion (KGC). PyTorch Geometric's MessagePassing framework is used in the code provided to build this framework, with a focus on relation-specific message modulation and simplicity.

2.2.1 Architecture and Relational Encoding

By including relation embeddings into the message passing process, the R-LightGCN model expands upon the regular LightGCN. The model's core component, the RLightGCNConvVar layer, substitutes a straightforward but effective aggregate for complex linear transformations and non-linearities. In the message function, the model makes the aggregation relation-specific by modulating the neighbor's embedding (x_j) using the relation embedding $r = R[r]$ through an element-wise product (Hadamard product), implemented in the code as $x_j * rel_{vec}$. This ensures the message transmitted is conditioned on the type of edge connecting the nodes. This relation-modulated message is then normalized by the standard symmetric term before being aggregated by addition (aggr = 'add'), maintaining the efficiency of the LightGCN framework.

Layer-wise Message Propagation Formula

The message $\mathbf{m}_{j \rightarrow i}$ sent from node j to node i over relation r is:

$$\mathbf{m}_{j \rightarrow i} = \text{Norm}_{ji} \cdot (\mathbf{x}_j^{(k)} \odot \mathbf{r})$$

where $\mathbf{r} = \mathbf{R}[r]$ is the relation embedding, and $\text{Norm}_{ji} = \frac{1}{\sqrt{|\mathcal{N}(i)| |\mathcal{N}(j)|}}$ is the symmetric normalization term.

The new embedding for node i in layer $k + 1$ is the sum of these messages:

$$\mathbf{x}_i^{(k+1)} = \sum_{j \in \mathcal{N}(i)} \mathbf{m}_{j \rightarrow i}$$

2.2.2 Model Structure and Layer Combination

One important design aspect of the R-LightGCN model is that relation embeddings are shared across all layers, enabling it to efficiently handle both node embeddings (self.embedding) and relation embeddings (self.rel_emb). There are K instances of R-LightGCNConv in the model's basic structure, which is represented by the self.convs module (where K is the number of layers). These convolutional layers are used iteratively to process the data in the encode technique. The final node embedding $\mathbf{z}$ is the outcome of a linear combination (average) of embeddings from every layer, including the initial embedding $\mathbf{x}(0)$ (referred to in the code as `x0` and `out`).

Final Node Embedding Combination Formula

The final embedding $\mathbf{z}$ for a node is the average of its embeddings from all K layers, including the initial embedding $\mathbf{x}^{(0)}$:

$$\mathbf{z} = \frac{1}{K + 1} \sum_{k=0}^K \mathbf{x}^{(k)}$$

2.2.3 Data Preprocessing and Decoding for Link Prediction

The code demonstrates key preprocessing steps for converting triples into a graph structure optimized for GNN training. This includes adding reverse edges (e.g., for each (h, r, t) triple, a corresponding (t, r, h) is added) to ensure bidirectional message passing, as well as incorporating self-loops with a unique relation ID (`num_relations`). Self-loops ensure each node's own current feature vector (starting from $x^{(0)}$) in the message aggregation process at every layer, so it does not get overwritten purely by neighbors' information.

To assess models performance in a link prediction task, the Dot Product Decoder or the DistMult Decoder is used to compute the score $S(h,r,t)$ for a given triple (implemented in the “decode” function in the code provided). Refer to Section 3.1.1 for the ablation study of R-LightGCN decoder.

2.3 Relational Graph Attention Network (R-GAT)

The Relational Graph Attention Network (R-GAT), often referred to in literature by variants like RAGAT or r-GAT, is an advanced Graph Neural Network (GNN) model designed to tackle the complexities of multi-relational graphs, particularly in Knowledge Graph Completion (KGC). It integrates the selective power of the attention mechanism into the process of aggregating information from neighbors, but critically, makes this attention process specific to the type of relation.

2.3.1 Architecture and Relational Attention

R-GAT extends the standard Graph Attention Network (GAT) by introducing relation-specific parameters into the attention mechanism. Unlike GAT, where the attention weights depend only on the features of the two connected entities, R-GAT dictates that different relations should lead to different message transformations and different levels of importance, addressing the heterogeneous nature of knowledge graphs. In an R-GAT layer, before aggregation, a linear transformation is typically applied to the neighbor's feature vector h_j . The key innovation is how the attention coefficient is calculated, incorporating the relation type r of the edge (j,i) . This enables the model to adaptively learn how relevant a neighbor j is to the central node i based on *why* they are connected. R-GAT often utilizes Multi-Head

Attention to increase the capacity of the model to learn various semantic aspects of the relationships, improving the stability of the attention learning process.

Relational Attention Coefficient Formula

The unnormalized attention score e_{ij}^r between node i and neighbor j connected by relation r is:

$$e_{ij}^r = \mathbf{a}^T \cdot \text{LeakyReLU}(\mathbf{W}_r \mathbf{h}_i \parallel \mathbf{V}_r \mathbf{h}_j)$$

where $\mathbf{h}_i$ and $\mathbf{h}_j$ are the input features (embeddings), $\mathbf{W}_r$ and $\mathbf{V}_r$ are relation-specific transformation matrices (or a single matrix if simplified), $\parallel$ denotes concatenation, and $\mathbf{a}$ is the relation-specific attention vector.

The normalized attention coefficient α_{ij}^r is computed using the **softmax** function over all neighbors k of node i :

$$\alpha_{ij}^r = \text{softmax}_j(e_{ij}^r) = \frac{\exp(e_{ij}^r)}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik}^r)}$$

2.3.2 Propagation and Multi-Head Aggregation

The final propagation step for node i involves using the normalized attention coefficients α_{ij}^r to compute a weighted sum of the transformed neighbor features, which are then passed through an activation function (σ). In a Multi-Head R-GAT layer, this process is replicated K times with independent parameter sets $(\mathbf{W}_k^r, \mathbf{V}_k^r, \mathbf{a}^k)$ to capture diverse relational semantics. The results from the K heads are then aggregated, typically by concatenation (for the first layer) or averaging (for the final layer) to yield the node's updated representation. This mechanism ensures that the aggregation process focuses on the most informative neighbors for each relation type, which is a key advantage over non-attentional GNNs like R-GCN or R-LightGCN.

2.3.3 Application and Decoding

R-GAT is primarily employed as an encoder within an encoder-decoder framework for KGC. The encoder's role is to generate high-quality, context-aware entity embeddings ($\mathbf{z}_i = \mathbf{h}_i'$) by leveraging the relational graph structure through its attention mechanism. Once the entity representations are learned, a separate decoder component is used to score the plausibility of

a given triple (h, r, t) . Common decoding methods include translation-based models (like TransE) or factorization-based models (like DistMult or ComplEx), similar to R-LightGCN. For instance, the final embeddings are often used in a scoring function $S(h, r, t)$ to predict the missing link. The use of R-GAT is thus an explicit attempt to leverage the structural heterogeneity of a knowledge graph to learn more expressive embeddings, leading to superior performance in ranking metrics like MRR and Hits@K.

General Decoding Score Formula

A common KGC scoring function $S(h, r, t)$ used with GNN embeddings:

$$S(h, r, t) = \psi(\mathbf{z}_h, \mathbf{r}, \mathbf{z}_t)$$

where $\mathbf{z}_h$ and $\mathbf{z}_t$ are the learned R-GAT entity embeddings, $\mathbf{r}$ is the relation embedding, and ψ is a scoring function (e.g., DistMult, which is $\mathbf{z}_h \odot \mathbf{r} \cdot \mathbf{z}_t$).

2.4 Relational Graph Isomorphism Network (R-GIN)

The Graph Isomorphism Network (GIN) is a powerful and theoretically grounded Graph Neural Network (GNN) designed to achieve maximal expressive power in distinguishing different graph structures. Unlike attention-based models such as R-GAT or lightweight aggregation models like R-LightGCN, GIN emphasizes injective aggregation and strong representational capacity through multi-layer perceptrons (MLPs). This architecture makes it well-suited for capturing fine-grained structural patterns in both homogeneous and relational graphs.

2.4.1 Architecture and Message Passing

At its core, GIN operates through iterative message passing across graph layers, where each node aggregates information from its neighbors and updates its own embedding using an MLP transformation. For a node i at layer $k + 1$, the update rule is given by:

$$x_i = \text{MLP}\left((1 + \epsilon)x_i + \sum_{j \in \mathcal{N}(i)} (x_j \odot r_{e_{ij}})\right)$$

where ϵ is a learnable scalar that controls the weighting between a node’s own features and those of its neighbors. The message $m_{j \rightarrow i}$ is typically a linear transformation of the neighbor’s embedding x_j^k , computed as:

$$\mathbf{m}_{j \rightarrow i} = W \mathbf{x}_j^{(k)}$$

Here, W is a shared learnable weight matrix across all nodes within a layer. The summation ensures that neighbor information is aggregated symmetrically, making GIN invariant to the order of neighbors, a key requirement for graph representation learning.

Unlike R-LightGCN, which employs simple element-wise modulation by relation embeddings, GIN relies on the expressive power of its MLPs to capture non-linear dependencies and higher-order structural relationships between nodes.

2.4.2 Relational Extensions

For multi-relational graphs, the standard GIN can be extended to R-GIN by introducing relation-aware message transformations. In this setup, each relation type r modulates the message using a learnable relation embedding r_{emb} :

$$\mathbf{m}_{j \rightarrow i}^{(r)} = W(\mathbf{x}_j^{(k)} \odot \mathbf{r}_{emb})$$

where $\odot$ denotes element-wise multiplication. This enables the model to differentiate how information is propagated across edges of different semantic types, making it suitable for Knowledge Graph Completion (KGC) tasks. Variants of R-GIN further expand this relational modeling capacity:

R-GIN (Embed): Employs direct embedding modulation $x_j^k \odot r_{emb}$ for lightweight relation awareness.

R-GIN (FiLM): Uses feature-wise linear modulation, where each relation generates scaling (γ_r) and shifting (β_r) parameters applied to Wx_j , allowing finer-grained relational conditioning.

R-GIN (LoRA): Incorporates Low-Rank Adaptation (LoRA), decomposing per-relation transformations as $W + \alpha A_r B_r^T$. This allows efficient, relation-specific adaptation without inflating parameter count, striking a balance between model expressiveness and scalability.

2.4.3 Layer Combination and Embedding Aggregation

After K layers of message passing, the final node representation $\mathbf{z}$ is obtained by averaging embeddings across all layers, including the initial input features $\mathbf{x}^0$:

$$\mathbf{z} = \frac{1}{K+1} \sum_{k=0}^K \mathbf{x}^{(k)}$$

This approach captures both local and global structural information by promoting feature propagation at different receptive field depths. It is conceptually similar to the layer combination in R-LightGCN, but the MLP-based updates provide deeper transformations at each layer.

2.4.4 Decoding for Link Prediction

For Knowledge Graph Completion or link prediction, GIN and its relational variants use decoder functions to compute the plausibility of a given triple (h, r, t) . Two common decoders are employed:

Dot Product Decoder: Computes relation-independent similarity between head and tail embeddings as:

$$S(h, r, t) = \mathbf{e}_h^\top \mathbf{e}_t$$

which assumes uniform relation semantics.

DistMult Decoder: Introduces a learnable relation-specific parameter vector w_r , defining

$$S(h, r, t) = (\mathbf{e}_h \odot \mathbf{w}_r \odot \mathbf{e}_t)^\top \mathbf{1}$$

enabling asymmetric and heterogeneous relation modeling.

The choice of decoder allows analysis of whether relational dependencies are primarily captured by the encoder or delegated to the scoring function, forming the basis for the decoder ablation experiments.

3. Experiments

3.1 Ablation Study on R-LightGCN

To understand the impact of individual architectural components in the R-LightGCN to the model’s ability to capture relational diversity, a series of ablation experiments are conducted.

3.1.1 Decoder Ablation: DistMult vs. Dot

Decoder ablation aims to evaluate how relation modeling in the scoring function affects model performance. It reveals if the encoder alone captures sufficient relational signal or if decoding needs to be relation-aware to rank links correctly. The dot product decoder is set as the default decoder in R-LightGCN computes the scores, $S(h, r, t)$ using a relation-modulated element-wise product between head and tail embeddings. The score is simply:

$$s(h, t) = \sum_{i=1}^d e_h^{(i)} \cdot e_t^{(i)} = \mathbf{e}_h^T \mathbf{e}_t$$

where $e_h, e_t \in \mathbb{R}^d$ represent the head and tail entity embedding (vector) respectively while d is the embedding dimension. Dot Product decoder is relation-agnostic and performs well when relations are essentially identical, such as phrases like "is similar to" or "is connected to,".

On the other hand, DistMult decoder, which can capture relation-specific asymmetry like “parent of”, “located in” is more expressive but could lead to overfitting issues, especially in sparse or low-diversity datasets. DistMult decoder introduces relation-specific multiplicative weights that scale each dimension differently:

$$s(h, r, t) = \sum_{i=1}^d e_h^{(i)} \cdot w_r^{(i)} \cdot e_t^{(i)} = \mathbf{e}_h^T \text{diag}(\mathbf{w}_r) \mathbf{e}_t$$

where $w_r \in \mathbb{R}^d$ represents the learnable relation-specific parameter vector.

3.1.2 Transformation Strategy: Scalar vs. Diagonal

Two methods for relation-specific neighbor embedding transformation were investigated to help determine whether relation-aware aggregation improves link prediction or solely introduces noise. First, element-wise multiplication is used to apply the Scalar Transformation, which represents each relation by a single scalar value. In order to enable dimension-wise modulation, we also experiment with diagonal transformation, in which each relation is represented by a diagonal matrix. While diagonal transformation gives more expressiveness, scalar transformation offers computational simplicity.

3.1.3 Normalization Scope: Global vs. Per-Relation

Normalization is used in R-LightGCN when aggregating messages from neighbour nodes to prevent biased representations where denser nodes dominate the embedding updates. This ensures that nodes with many connections (high degree) do not overwhelm the representation of nodes with fewer connections.

Instead of normalizing per relation type, global normalisation treats the entire graph as a unified structure, meaning all edges regardless of their relation type are normalized using the same rule. This is especially useful when relation types are semantically identical, as it prevents In our implementation, the symmetric normalization factor is computed as follow:

$$norm(i, j) = \frac{1}{\sqrt{deg(i) \cdot deg(j)}}$$

where deg represents node degree, i and j are connected nodes.

Per-relation normalisation is done separately for each relation type, allowing different relations to have their own local degree distributions. The normalisation factor is:

$$norm(i, j) = \frac{1}{\sqrt{deg_r(i) \cdot deg_r(j)}}$$

where $deg_r(i)$ is the degree of node i within relation r only. If we normalize per relation, each relation’s messages are scaled relative to how many connections exist within that relation.

3.1.4 Results and Discussion

Dataset 1: WN18RR

Table 2: Test Results on WN18RR dataset for LightGCN and R-LightGCN Variants

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
LightGCN	0.9319	0.5994	0.8336	0.8805
R-LightGCN (Baseline)				
Scalar + Global + DotProduct	0.9394	0.6396	0.8213	0.8674
Relation Transform Ablation				
Diagonal + Global + DotProduct	0.9377	0.6361	0.8253	0.8690
Decoder Ablation				
Scalar + Global + DistMult	0.9160	0.5444	0.7648	0.8162
Normalization Ablation				
Scalar + Per-relation + DotProduct	0.9400	0.6270	0.8232	0.8703
Other Variants				
Scalar + Per-relation + DistMult	0.8992	0.3623	0.6825	0.7837
Diagonal + Per-relation + DistMult	0.9096	0.4564	0.7371	0.8015

Table 2 shows the performances of trained LightGCN, Baseline R-LightGCN and its variants on the test data. The LightGCN model achieves strong performance (AUC = 0.9319, Hits@10 = 0.8805), showing that simple neighborhood aggregation already captures much of the graph structure. For Relation Transform ablation, the R-LightGCN variant (Diagonal + Global + DotProduct) performs similarly to the baseline, implying that a more expressive diagonal transform adds little benefit for WN18RR’s limited relation diversity. Normalization ablation (Scalar + Per-relation + DotProduct) with a slight AUC gain = 0.9400 but no improvement in Hits@k, suggesting global normalization is sufficient and more stable.

An interesting insight gained from the decoder ablation is that the baseline R-LightGCN model (Scalar + Global + DotProduct) outperforms all its variants for all the evaluation metrics. As shown in Figure 5, loss falls quickly then plateaus around Loss=0.666, indicating convergence after 15-20 epochs. We realize one of the possible contributing factors could be

due to the decoder variant. Variants with DistMult decoder have generally lower AUC score and Hits@10. The addition of a complex decoder such as DistMult is inconsistent with the simplicity of the encoder, resulting in lower generalization and unstable optimization. The DistMult decoder introduces relation-specific weights for scoring, assuming the encoder has already captured rich relational semantics, yet, R-LightGCN which uses uniform neighbor aggregation, does not learn deep relation-specific transformation.

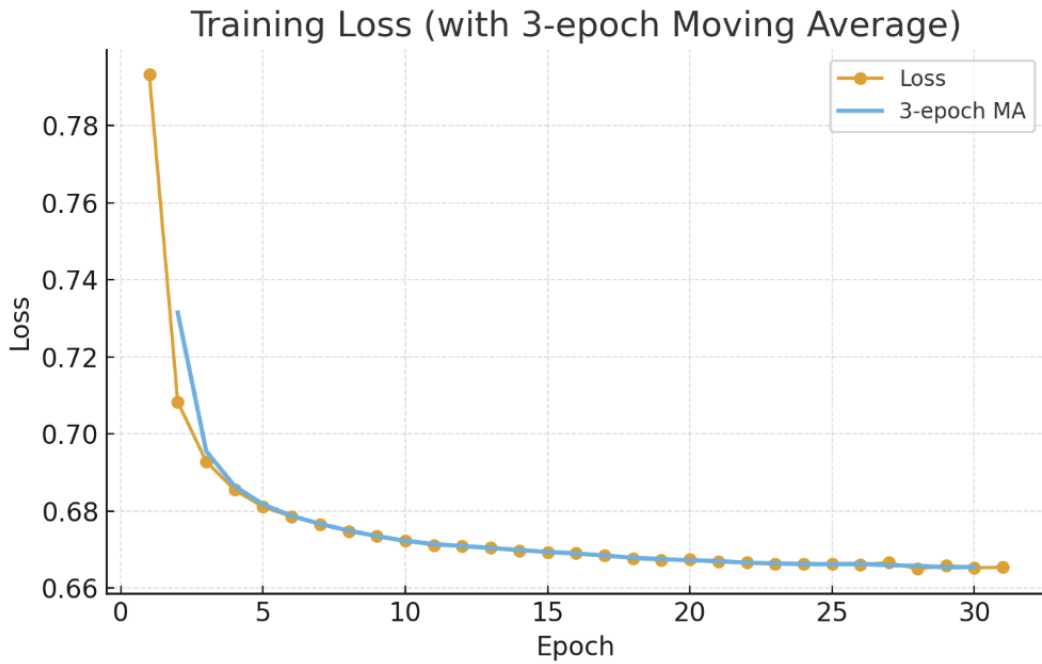


Figure 5: Loss Curve for R-LightGCN Model (Scalar + Global + DotProduct)

Dataset 2: FB15K-237

Table 3: Test Results on FB15k-237 dataset for LightGCN and R-LightGCN Variants

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
LightGCN	0.8821	0.2720	0.5579	0.7034
R-LightGCN (Baseline)				
Scalar + Global + DotProduct	0.8877	0.1750	0.4941	0.6641
Relation Transform Ablation				
Diagonal + Global + DotProduct	0.8893	0.1716	0.4935	0.6651
Decoder Ablation				
Scalar + Global + DistMult	0.9563	0.1373	0.3533	0.4903
Normalization Ablation				
Scalar + Per-relation + DotProduct	0.8883	0.1677	0.4896	0.6579
Other Variants				
Scalar + Per-relation + DistMult	0.9529	0.1642	0.4150	0.5566
Diagonal + Per-relation + DistMult	0.9528	0.1043	0.2875	0.4230

From Table 3, we observe that the base model (LightGCN) performs best overall with AUC = 0.8821 and Hits@10 = 0.7034. FB15K-237 was designed to remove inverse and duplicate relations, making the graph sparser and has semantically noisier relations. LightGCN, which treats all neighbors equally, works well with the FB15K-237 dataset as it focuses purely on structural connectivity. The downward trend of the loss curve in Figure 6 further proves this.

Unlike R-GAT models which encoder can extract signals even from weak or sparse relations (with Hits@10 = 0.9734 for Embed-Relation + DistMult R-GAT variant as shown in Table 6), R-LightGCN variants which add relation modulation (scalar or diagonal) fail to capture the weak relation signals in the FB15K-237 dataset with Hits@10 between 0.4230 to 0.6651.

With the encoder’s limited relational expressiveness in R-LightGCN, added complexity like DistMult decoder or per-relation normalization does not pay off. Another finding is that, when employing the DistMult decoder, there is a divergence between AUC and Hits@k, with an increased AUC accompanied by a poorer ranking performance. This suggests that while the model performs well at classification, it is not very good at ordering them. Therefore, choosing the right metrics and aligning the architecture are essential for link prediction.

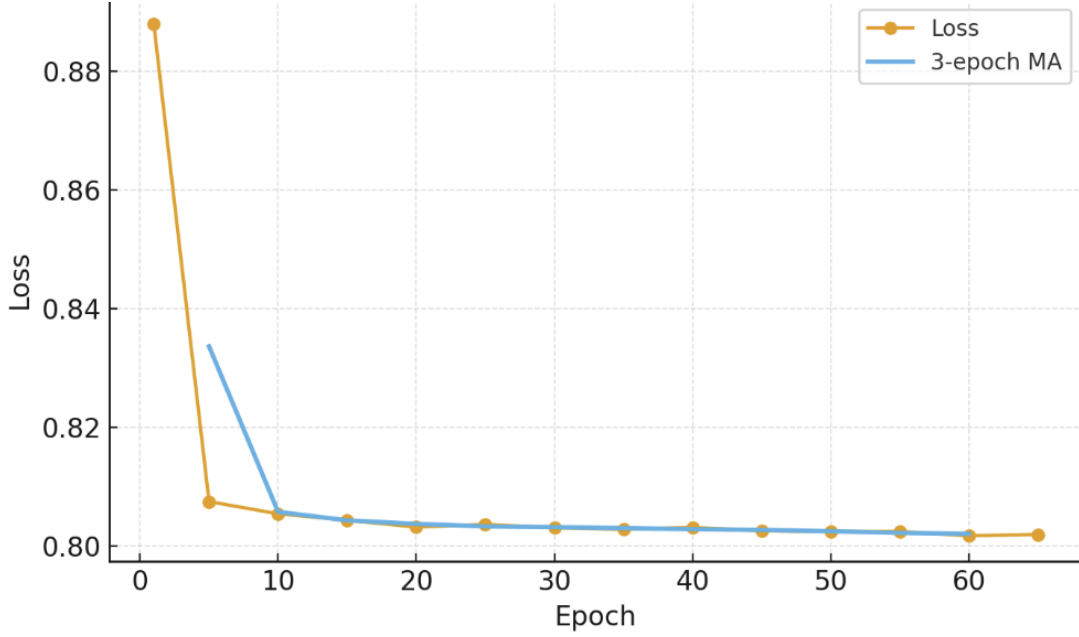


Figure 6: Loss Curve for R-LightGCN Model (Scalar + Global + DotProduct)

Dataset 3: CORA

Table 4: Test Results on CORAdataset for LightGCN and R-LightGCN Variants

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
LightGCN	0.8768	0.3567	0.6214	0.7068
R-LightGCN (Baseline)				
Scalar + Global + DotProduct	0.8610	0.3416	0.6233	0.7135
Relation Transform Ablation				
Diagonal + Global + DotProduct	0.8699	0.3435	0.6110	0.6983
Decoder Ablation				
Scalar + Global + DistMult	0.8038	0.2419	0.4658	0.5674
Normalization Ablation				
Scalar + Per-relation + DotProduct	0.8664	0.3368	0.6044	0.7040
Other Variants				
Scalar + Per-relation + DistMult	0.7552	0.2125	0.4478	0.5398
Diagonal + Per-relation + DistMult	0.7964	0.2163	0.4573	0.5721

Table 4 displays the test results of the LightGCN and R-LightCGN variants trained on the CORA dataset. Again, the base model (LightGCN) achieved the highest overall ranking performance (AUC = 0.8768, Hits@10 = 0.7068), showing a steady decline from 1.00 to 0.67

as seen in Figure 7. This confirms that simple structural aggregation can effectively capture homophily in the CORA citation network.

In addition, since CORA has only one edge type, citations, incorporating relation-specific transformations or normalization does not add meaningful information to the message-passing process. Models like R-LightGCN were originally designed for multi-relational graphs (e.g., WN18RR or FB15k-237), where relation embeddings play a key role and do not gain advantage. This reinforces that simpler architectures (like LightGCN) are better suited for homogeneous networks, while complex relation-aware designs benefit primarily from heterogeneous graphs.

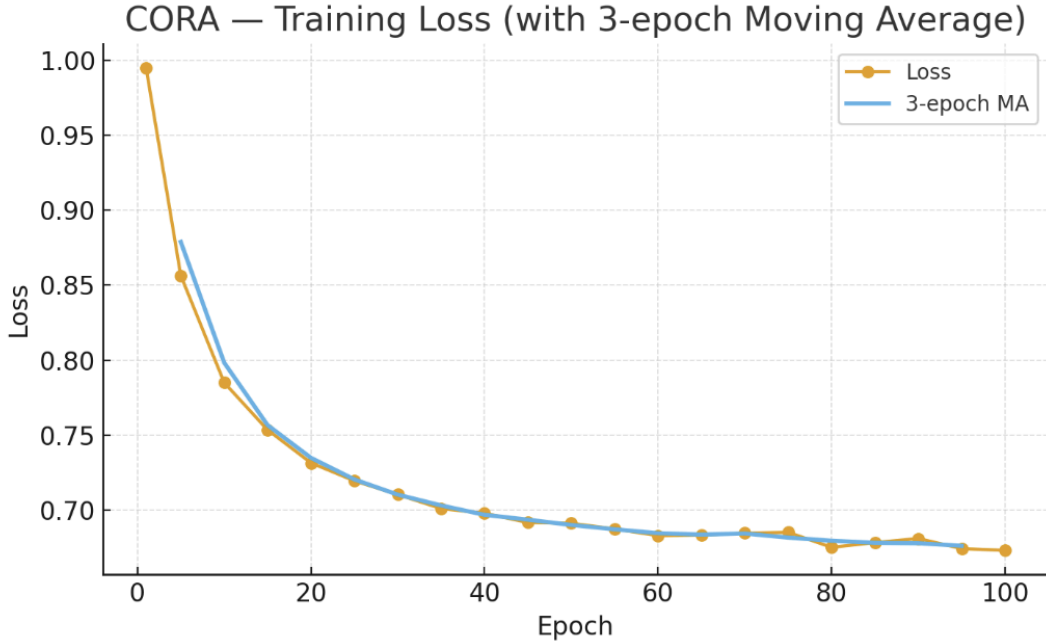


Figure 7: Loss Curve for R-LightGCN Model (Scalar + Global + DotProduct)

3.2 Ablation Study on R-GAT

To evaluate the contribution of each individual component in the Relational Graph Attention Network, we conducted a comprehensive ablation study to examine both the encoder and decoder. We investigate (1) the impact of relation-aware scoring functions by comparing DistMult and DotProduct decoders, and (2) the effectiveness of different relation modeling by comparing Attention per Relation and Embed Relation encoders.

3.2.1 Decoder Ablation: DistMult vs. DotProduct

Unlike R-LightGCN which performs message passing with uniform neighbor aggregation, RGAT utilizes relation-specific GATs to capture relational dependencies during the encoding. With each relation type modeled with its own attention mechanism, it is able to learn distinct patterns between entities under different relations. Since the encoder already encodes relational information through attention, we compare two decoder variants to investigate the necessity of relation-aware scoring function during decoding. The first variant uses the DistMult decoder, which performs relation-aware bilinear scoring between head and tail entities. The score is computed as:

$$s(h, r, t) = \sum_{i=1}^d e_h^{(i)} \cdot w_r^{(i)} \cdot e_t^{(i)} = \mathbf{e}_h^T \text{diag}(\mathbf{w}_r) \mathbf{e}_t$$

where $e_h, e_t \in \mathbb{R}^d$ denote the embeddings of the head and tail entities generated by the relational attention encoder, and $w_r \in \mathbb{R}^d$ represents the learnable relation-specific parameter vector.

On the other hand, the Dot Product decoder omits relation-specific parameters and computes similarity solely between entity embeddings:

$$s(h, t) = \sum_{i=1}^d e_h^{(i)} \cdot e_t^{(i)} = \mathbf{e}_h^T \mathbf{e}_t$$

By comparing performance between these two formulations, we can determine the degree to which RGAT’s encoder captures relational dependencies, and whether relation-aware decoding is necessary.

3.2.2 Encoder Ablation: Attention per Relation vs. Embed Relation

We examine two approaches for integrating relational information within graph attention networks. The first approach, Attention-per-Relation Encoder, assigns a distinct attention mechanism to each relation type. This allows the model to learn separate transformation weights and attention patterns for every relation. The second approach, Embed-Relation Encoder, replaces these multiple relation-specific layers with a single shared attention mechanism to incorporate relation embeddings to represent relational context. This results in

significantly reducing the computational cost and model size and utilizing the learned embeddings to differentiate between relation types.

3.2.3 Results and Discussion

Dataset 1: WN18RR

Table 5: Test Results on WN18RR dataset for GAT and RGAT Variants

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
GAT	0.8011	0.4145	0.5648	0.6206
RGAT (Baseline)				
Attention-per-Relation + DistMult	0.9128	0.5964	0.7237	0.7843
Decoder Ablation				
Attention-per-Relation + DotProduct	0.7422	0.2731	0.4052	0.4828
Encoder Ablation				
Embed-Relation + DistMult	0.9224	0.5520	0.7610	0.8251
Other Variants				
Embed-Relation + DotProduct	0.8052	0.4199	0.5453	0.5973

The baseline RGAT model (Attention-per-Relation + DistMult) in Table 5 demonstrated improvements over the base GAT model, achieving an AUC of 0.9128 compared to 0.8011, and a Hits@10 of 0.7843 compared to 0.6206. This indicates that relation attention is beneficial for WN18RR, which contains hierarchical and asymmetric relations. On the other hand, GAT treats all edges uniformly, which limits the model's ability to capture directional semantics. The decoder ablation further illustrates the importance of relation-aware scoring. When replacing DistMult with the simpler DotProduct decoder, the performance drops significantly to an AUC of 0.7422 and Hits@10 of 0.4828. This 36% decrease in Hits@10 demonstrates that even with relation-specific attention in the encoder, the decoder must still account how each relation functions to effectively discriminate between different relation patterns.

Interestingly, the encoder ablation (Embed-Relation + DistMult) achieves the best overall performance with an AUC of 0.9224 and Hits@10 of 0.8251, which slightly outperforms the baseline RGAT. As shown in Figure 8, the loss displayed improvement from 1.15 to 0.07 within 16 epochs. This variant uses a unified attention mechanism with relation embeddings

rather than separate attention parameters per relation. This suggests that parameter sharing can reduce overfitting and capture relational patterns effectively. However, the combined variant (Embed-Relation + DotProduct) shows moderate performance with an AUC of 0.8052 and Hits@10 of 0.5973. This indicates that the relation embeddings in the encoder cannot fully compensate for the lack of relation-aware scoring in the decoder. Therefore, these results highlight that integrating relation information in both encoder and decoder components yields the best performance for WN18RR.

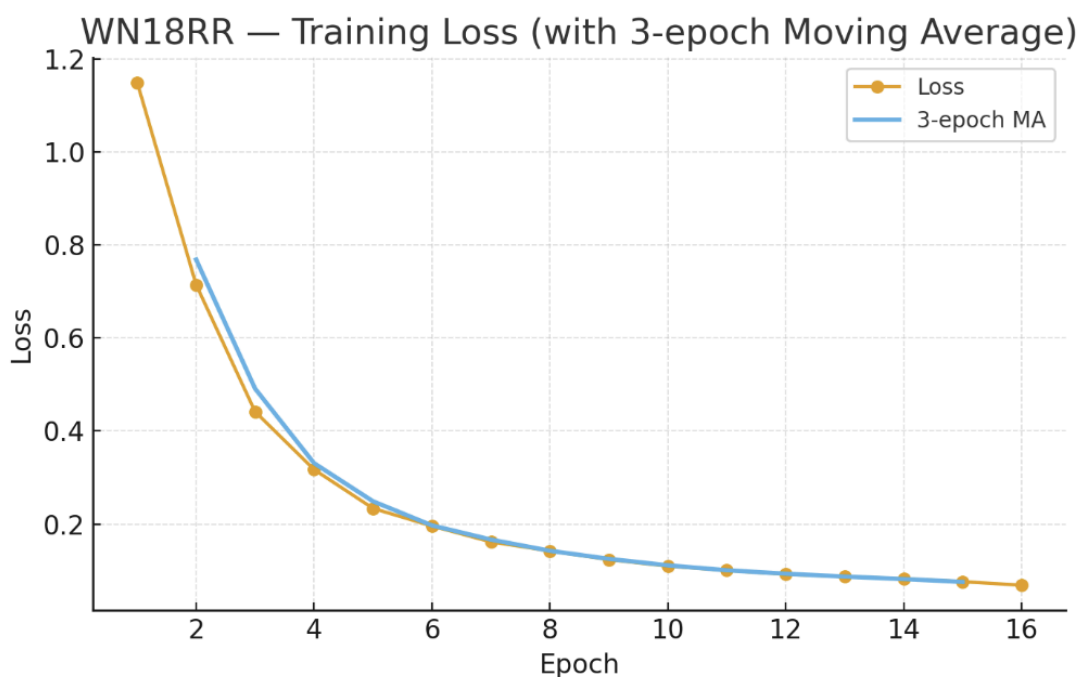


Figure 8: Loss Curve for Encoder Ablation (Embed-Relation + DistMult)

Dataset 2: FB15K-237

Table 6: Test Results on FB15k-237 dataset for GAT and RGAT Variants

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
GAT	0.8495	0.2976	0.5930	0.6974
RGAT (Baseline)				
Attention-per-Relation + DistMult	0.9861	0.6703	0.9181	0.9708
Decoder Ablation				
Attention-per-Relation + DotProduct	0.8923	0.3150	0.6341	0.7460
Encoder Ablation				
Embed-Relation + DistMult	0.9870	0.6824	0.9224	0.9734
Other Variants				
Embed-Relation + DotProduct	0.9040	0.3060	0.6333	0.7521

Table 6 shows the baseline RGAT model (Attention-per-Relation + DistMult) achieving exceptional performance on FB15K-237. It obtained an improvement in AUC of 0.9861 and Hits@10 of 0.9708 as compared to the base GAT model (AUC 0.8495, Hits@10 0.6974). This demonstrates the effectiveness of relation-specific attention mechanisms on this dataset, which contains diverse relation types from the Freebase knowledge base. The relation attention structure allows the model to assign different importance weights to neighbors based on edge type, such that less informative relations do not contribute as much as higher predictive ones. The decoder ablation further confirms the importance of relation-aware scoring. Switching to a DotProduct decoder with relation-specific attention results in a drop of AUC to 0.8923 and Hits@10 to 0.7460. This indicates that the decoder must also account how each relation functions similarly to the previous dataset.

The encoder ablation (Embed-Relation + DistMult) achieves the highest performance overall with an AUC of 0.9870 and Hits@10 of 0.9734. Figure 9 shows the downward trend of the loss. This suggests that directly embedding relation information into node messages to separate attention parameters per relation results in better generalization on FB15K-237's relation patterns. However, the variant (Embed-Relation + DotProduct) shows reduced performance with an AUC of 0.9040 and Hits@10 of 0.7521. This is similar to the pattern observed in WN18RR, that relation embeddings in the encoder contribute to improved results, but pairing them with a relation-aware decoder like DistMult is crucial for achieving optimal results.

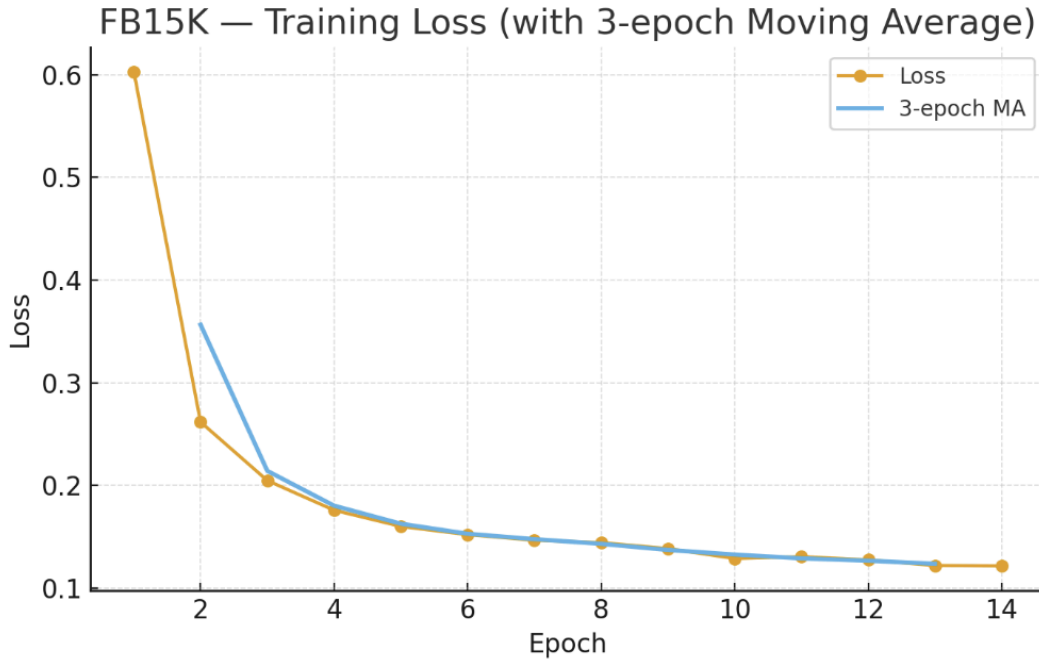


Figure 9: Loss Curve for Encoder Ablation (Embed-Relation + DistMult)

Dataset 3: CORA

Table 7: Test Results on CORA dataset for GAT and RGAT Variants

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
GAT	0.7160	0.1670	0.3387	0.4279
RGAT (Baseline)				
Attention-per-Relation + DistMult	0.8044	0.3036	0.5598	0.6243
Decoder Ablation				
Attention-per-Relation + DotProduct	0.7583	0.2306	0.4383	0.5066
Encoder Ablation				
Embed-Relation + DistMult	0.7879	0.3245	0.5541	0.6063
Other Variants				
Embed-Relation + DotProduct (Variant)	0.7215	0.1717	0.3235	0.4165

Table 7 shows the test results for GAT and RGAT variants on the CORA dataset, a homogeneous graph where all edges represent the same "cites" relation. Despite this lack of relational diversity, the baseline RGAT model (Attention-per-Relation + DistMult) showed improvements over the base GAT, achieving an AUC of 0.8044 compared to 0.7160, and a Hits@10 of 0.6243 compared to 0.4279. This improvement suggests that relation attention

can still enhance performance in single relation graphs by adaptively weighting neighbors based on structural importance. The decoder ablation further confirms that relation-aware scoring remains beneficial even in homogeneous settings, as switching to a DotProduct decoder reduces performance to an AUC of 0.7583 and Hits@10 of 0.5066.

The encoder ablation (Embed-Relation + DistMult) achieves results with an AUC of 0.7879 and Hits@10 of 0.6063, supported by Figure 10. This suggests that directly embedding relation information can be as effective as separating attention parameters per relation. The variant (Embed-Relation + DotProduct) shows the lowest performance among RGAT variants with an AUC of 0.7215 and Hits@10 of 0.4165. These results indicate that the combination of attention-based message passing and relation-aware decoding improves performance even if CORA's homogeneous structure limits the use of multi-relational mechanisms.

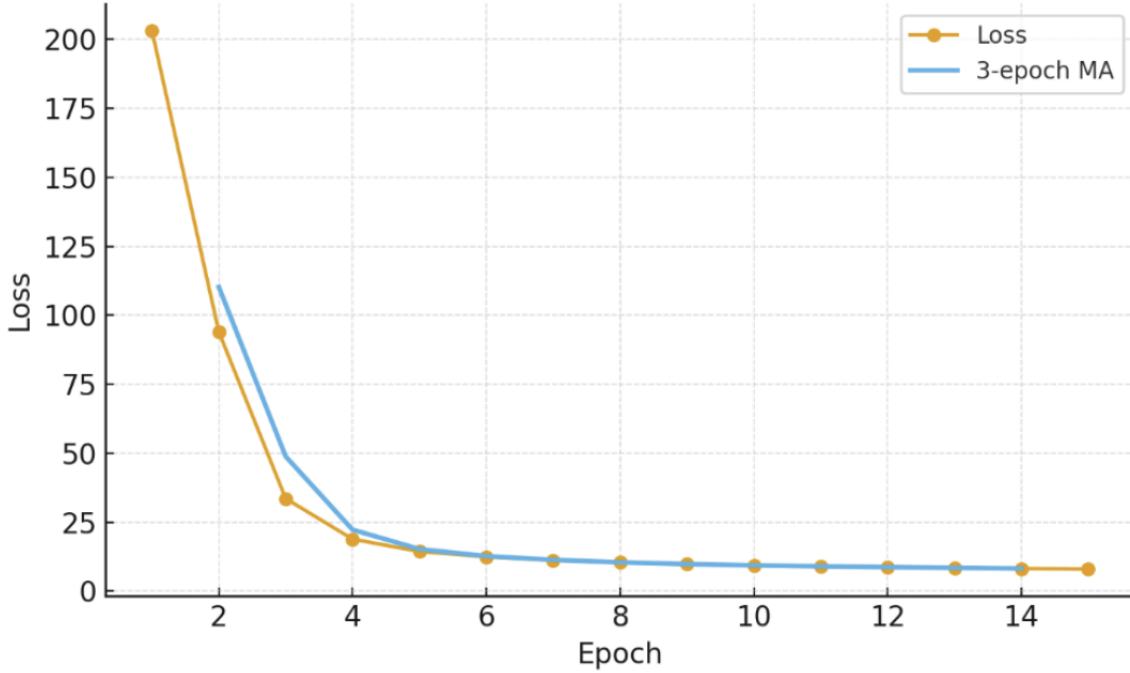


Figure 10: loss curve for RGAT model (Attention-per-Relation + DistMult)

3.3 Ablation Study on R-GIN

We ablate four message-passing variants to quantify how relation modeling affects link prediction: **Base GIN** (no relation signals), **R-GIN (embed relations)**, **R-GIN (FiLM)**, and **R-GIN (LoRA adapters)**. Across variants we keep the GIN update form and decoder fixed when isolating each factor.

Let $h_i \in R^d$ be the embedding of node i , N_i its neighbors, and r_{ij} the relation type on edge $j \rightarrow i$. All variants use the GIN update:

$$h_i = \text{MLP} \left((1 + \epsilon) h_i + \sum_{j \in \mathcal{N}(i)} m_{j \rightarrow i} \right)$$

where $m_{j \rightarrow i}$ is the relation-dependent (or independent) message.

To set a baseline of comparison, we create a base model where the different relation types are treated equally — messages passed are not altered by relation type. We ignore edge types and aggregate neighbor content only:

$$m_{j \rightarrow i}^{\text{Base}} = W h_j.$$

3.3.1 Decoder Ablation: DistMult vs. Dot

Unlike R-GAT or R-LightGCN, which naturally incorporate relational information into their message-passing process, the standard **GIN** encoder operates on homogeneous graphs and aggregates neighbor messages uniformly through summation. Since relational information in GIN is not explicitly encoded in the message-passing stage, the choice of decoder becomes crucial for determining how relation semantics are represented during link prediction.

To investigate this, we compared two decoder formulations, **DistMult** and **Dot Product**, on top of the same GIN encoder.

Dot Product Decoder (Relation-Independent)

The Dot Product decoder computes scores purely based on the similarity between the head and tail node embeddings:

$$S(h, r, t) = e_h^\top e_t$$

where $e_h, e_t \in R^d$ are the node embeddings produced by the GIN encoder.

This formulation omits any relation-specific parameters, treating all edges as equally weighted connections. It is most effective when relation types are homogeneous or when relational distinctions can be inferred indirectly from the structural patterns captured by the encoder.

DistMult Decoder (Relation-Aware)

The DistMult decoder introduces a learnable embedding vector $w_r \in R^d$ for each relation, computing:

$$S(h, r, t) = (e_h \odot w_r \odot e_t)^\top \mathbf{1}$$

This relation-aware bilinear form allows different relation types to modulate the interaction between entity embeddings. Each dimension of the embedding is scaled by the corresponding entry in w_r , enabling asymmetric or semantically diverse relations (e.g., “*parent of*”, “*located in*”) to be modeled more effectively.

In the context of GIN, this decoder compensates for the encoder’s lack of built-in relation awareness, effectively shifting relational modeling to the decoding stage.

Interpretation

By comparing the two, we assess whether the GIN encoder can implicitly capture relational dependencies through structural aggregation alone, or if explicit relation-aware decoding (as in DistMult) is essential for link prediction accuracy. Typically, the Dot Product variant serves as a control for purely structure-based reasoning, while DistMult enhances expressiveness by integrating semantic information that GIN does not inherently encode.

3.3.2 Encoder Ablation: Relation Encoding Strategies

To understand how different relation-aware encoders contribute to representation quality, we performed an ablation study on several GIN-based relational variants. Each variant modifies the message transformation mechanism in the GIN convolution layer while keeping the overall training setup identical. These experiments aim to evaluate the relative effectiveness of simple relation embeddings, feature-wise modulation (FiLM), and low-rank adaptation (LoRA) in capturing heterogeneous relational semantics for link prediction.

3.3.2.1 Base GIN (Homogeneous Baseline)

The **Base GIN** serves as a control model that performs standard Graph Isomorphism Network (GIN) message passing without explicit relation awareness. Each message is computed as:

$$m_{j \rightarrow i} = Wx_j$$

$$x_i = \text{MLP}\left((1 + \epsilon)x_i + \sum_j m_{j \rightarrow i}\right)$$

where W is a shared linear transformation, and ϵ is a learnable scalar. This baseline allows us to isolate the benefit of introducing relation-specific transformations.

3.3.2.2 R-GIN (Relation Embedding Modulation)

In **R-GIN-Embed**, each relation type r is assigned a learnable embedding vector r_{emb} . Messages are modulated through elementwise interaction between the neighbour feature x_j and its corresponding relation embedding:

$$m_{j \rightarrow i} = (\mathbf{x}_j \odot \mathbf{r}_{emb})$$

followed by the same GIN update rule.

This method introduces lightweight relation awareness, allowing the model to learn how different edge types influence feature propagation. It captures basic relational semantics while maintaining computational simplicity.

3.3.2.3 R-GIN-FiLM (Feature-wise Linear Modulation)

The **FiLM** variant draws inspiration from conditional normalisation methods. For each relation, two small MLPs generate scaling and shifting parameters (γ_r, β_r) that modulate the transformed neighbor embeddings:

$$m_{j \rightarrow i} = \gamma_r \odot (\mathbf{W} \mathbf{x}_j) + \beta_r$$

$$\mathbf{x}'_i = \text{MLP}\left((1 + \epsilon) \mathbf{x}_i + \sum_j m_{j \rightarrow i}\right)$$

This provides dimension-wise conditioning of node updates on the relation type, enabling finer-grained control than simple embedding modulation. FiLM introduces more parameters but achieves higher expressiveness, particularly when the same node participates in multiple types of relations.

3.3.2.4 R-GIN-LoRA (Low-Rank Adaptation)

Finally, the **R-GIN-LoRA** integrates **Low-Rank Adaptation (LoRA)** within the message transformation. Instead of a full relation-specific weight matrix, each relation uses a low-rank decomposition $A_r B_r^T$, where $A_r, B_r \in \mathbb{R}^d \times k$ and $k \ll d$:

$$m_{j \rightarrow i} = (\mathbf{W} + \alpha \mathbf{A}_r \mathbf{B}_r^T) \mathbf{x}_j$$

This formulation allows relation-specific fine-tuning in a parameter-efficient manner, balancing flexibility and scalability.

(a) LoRA-Per-Relation

In the LoRA-Per-Relation variant, each relation r maintains its own pair of low-rank matrices (A_r, B_r) . This enables relation-specific modulation of message transformations, allowing the model to learn distinct interaction patterns across different relation types. The per-relation decomposition improves expressiveness and captures relational asymmetry effectively, as each edge type can adapt the shared transformation W to better encode its semantics. However, this approach incurs slightly higher memory costs since each relation has its dedicated adapter.

(b) Unified LoRA

The Unified LoRA variant uses a single shared low-rank adapter across all relations, rather than maintaining individual A_r, B_r pairs. This design significantly reduces the total parameter count and ensures better generalization across sparse or unseen relations. While it sacrifices some relation-specific expressivity, the unified structure achieves a favorable trade-off between scalability and performance, especially in large-scale graphs with many relation types but limited per-relation samples.

3.3.3 Results and Discussion

Dataset 1: WN18RR

Table 8: Test Results on WN18RR dataset for GIN and R-GIN

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
GIN + DotProduct	0.8225	0.3141	0.6121	0.7154
RGIN (Baseline)				
Relation-Embedding + DotProduct	0.7884	0.1439	0.3866	0.5529
Decoder Ablation				
Relation-Embedding + DistMult	0.8011	0.2205	0.4781	0.5897
Encoder Ablation				
FiLM + DistMult	0.7955	0.3365	0.5226	0.6045
LoRA-Per-Relation + DistMult	0.8804	0.3957	0.6767	0.7581
Other Variant				
Unified LoRA + DistMult	0.8615	0.3286	0.6193	0.7202

Table 8 presents the test performance of various GIN-based models on WN18RR. The transition from the base GIN (AUC 0.8225, Hits@10 0.7154) to R-GIN, which incorporates relation embeddings, highlights the importance of relational context in improving link prediction. While the base model with the DotProduct decoder treats all relations uniformly, R-GIN introduces relation-specific embeddings that help differentiate edge types, though its performance remains limited by the simplicity of the DotProduct decoder (AUC 0.7884, Hits@10 0.5529). When the relation-aware DistMult decoder is added in the Decoder Ablation model, performance improves (AUC 0.8011, Hits@10 0.5897), showing that both relation-sensitive encoding and decoding are necessary for effectively capturing semantic variety in relational graphs.

The Encoder Ablation results further reveal the benefits of parameter-efficient adaptations. The FiLM variant improves over simple relation embeddings (AUC 0.7955) by modulating features per relation, while the LoRA-Per-Relation encoder achieves the highest scores (AUC 0.8804, Hits@10 0.7581), demonstrating the strength of low-rank adaptation in learning expressive yet efficient relational transformations. The Unified LoRA variant maintains strong performance (AUC 0.8615, Hits@10 0.7202), highlighting LoRA’s scalability and generalization as shown in Figure 11. Overall, these findings suggest that integrating relation-aware mechanisms, particularly LoRA-based encoders with DistMult decoders, offers a robust and efficient approach for modeling relational structures in complex knowledge graphs.

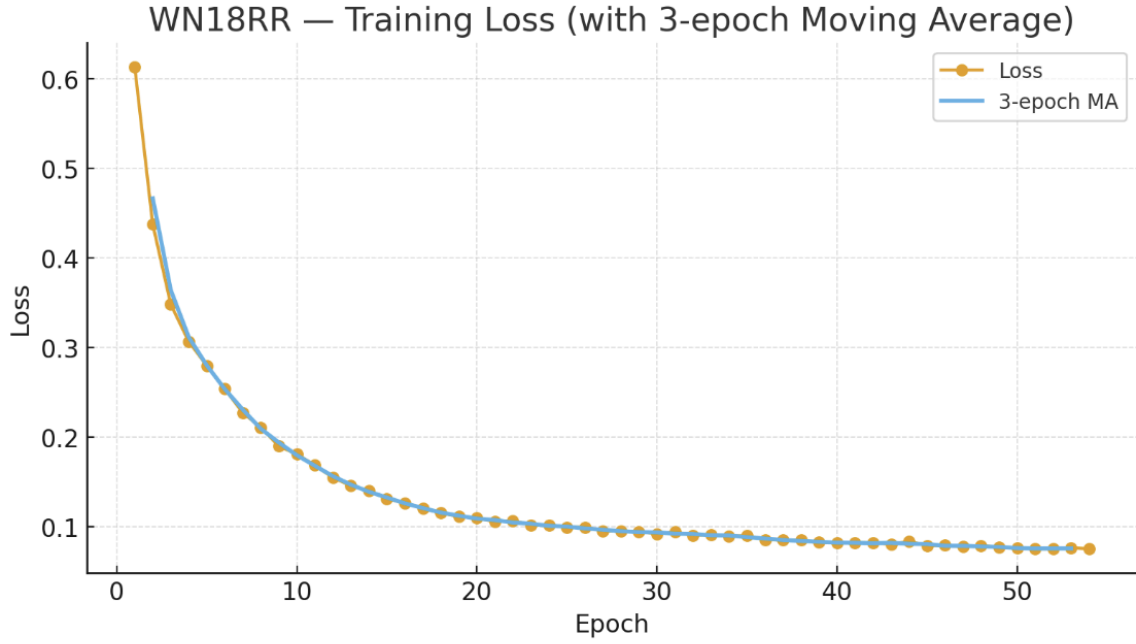


Figure 11: Loss Curve for LoRA-based Encoders with DistMult Decoders

Dataset 2: FB15K-237

Table 9: Test Results on FB15K-237 dataset for GIN and R-GIN

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
GIN + DotProduct	0.8863	0.2396	0.5613	0.7098
RGIN (Baseline)				
Relation-Embedding + DotProduct	0.8787	0.2292	0.5601	0.7082
Decoder Ablation				
Relation-Embedding + DistMult	0.9801	0.5358	0.8311	0.9233
Encoder Ablation				
FiLM + DistMult	0.9780	0.5247	0.8210	0.9120
LoRA-Per-Relation + DistMult	0.9802	0.5499	0.8408	0.9256
Other Variant				
Unified LoRA + DistMult	0.9796	0.5429	0.8362	0.9259

Table 9 presents the results of GIN and R-GIN variants on the FB15K-237 dataset, demonstrating the clear benefits of incorporating relation-aware mechanisms. The base GIN model (AUC 0.8863, Hits@10 0.7098) performs reasonably well but lacks sensitivity to different relation types. When relation embeddings are introduced in R-GIN (AUC 0.8787,

Hits@10 0.7082), the model becomes more capable of distinguishing between heterogeneous connections, though the simple DotProduct decoder still limits its relational expressiveness. Once the relation-aware DistMult decoder is used in the Decoder Ablation variant, performance increases sharply (AUC 0.9801, Hits@10 0.9233), showing that decoding relations bilinearly allows the model to scale and differentiate link patterns more effectively within FB15K-237’s dense relational structure.

The Encoder Ablation results further confirm the value of parameter-efficient relational adaptation. The FiLM-based model (AUC 0.9780) already provides strong modulation across relations, but LoRA-Per-Relation + DistMult slightly surpasses it (AUC 0.9802, Hits@10 0.9256) by achieving similar expressiveness with fewer parameters. LoRA’s low-rank adaptation enables efficient learning of relation-specific transformations without increasing model size, while the Unified LoRA variant maintains competitive accuracy (AUC 0.9796, Hits@10 0.9259). Overall, these results highlight that combining LoRA encoders with the DistMult decoder delivers high relational sensitivity and scalability, making it an effective strategy for complex knowledge graphs such as FB15K-237, as shown in the loss plot in Figure 12.

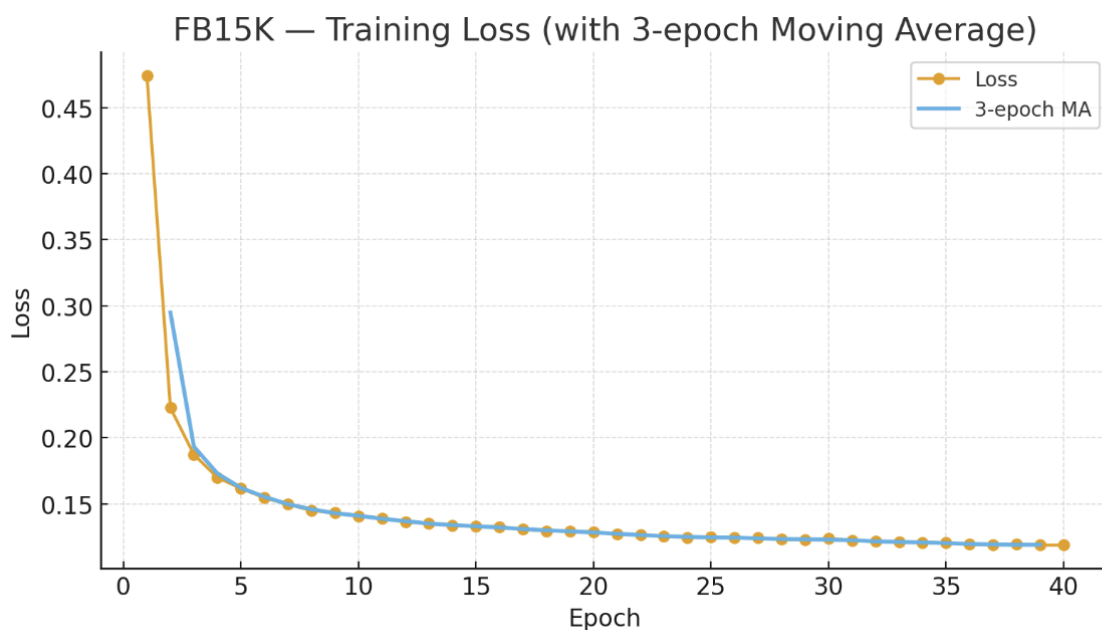


Figure 12: Loss Curve for LoRA-based Encoders with DistMult Decoders

Dataset 3: CORA

Table 10: Test Results on CORA dataset for GIN and R-GIN

Model	AUC	Hits@1	Hits@5	Hits@10
Base Model				
GIN + DotProduct	0.8049	0.1361	0.3861	0.5361
RGIN (Baseline)				
Relation-Embedding + DotProduct	0.5862	0.0797	0.1954	0.2818
Decoder Ablation				
Relation-Embedding + DistMult	0.7228	0.0759	0.2457	0.4080
Encoder Ablation				
FiLM + DistMult	0.6988	0.0882	0.2941	0.4383
LoRA-Per-Relation + DistMult	0.7122	0.0977	0.3235	0.4753
Other Variant				
Unified LoRA + DistMult	0.7063	0.1518	0.3757	0.5209

Table 10 presents the results of GIN and R-GIN models on the CORA dataset. The base GIN model (AUC 0.8049, Hits@10 0.5361) performs reasonably well given its uniform message aggregation, but it lacks sensitivity to relation types. When relation embeddings are introduced in R-GIN (AUC 0.5862, Hits@10 0.2818), the model struggles to generalize effectively, suggesting that relation embeddings alone are insufficient without a more expressive decoder. This performance drop indicates that while relation-awareness helps encode structural diversity, the limited capacity of the DotProduct decoder constrains how relational information is utilized within the model.

The ablation studies show that adding the DistMult decoder improves relational expressiveness. The Relation-Embedding + DistMult variant achieves better overall scores (AUC 0.7228, Hits@10 0.4080), showing the importance of relation-specific scoring in the decoding phase. Further enhancements in the encoder through FiLM and LoRA modules highlight the role of efficient modulation mechanisms. Among these, LoRA-Per-Relation + DistMult performs best (AUC 0.7122, Hits@10 0.4753), indicating that low-rank relational adaptations improve representation quality without excessive parameter growth. Collectively, these findings reinforce that integrating relation-aware decoding and scalable relational encoding, especially through LoRA, leads to stronger performance in link prediction on heterogeneous graph data, shown in Figure 13.

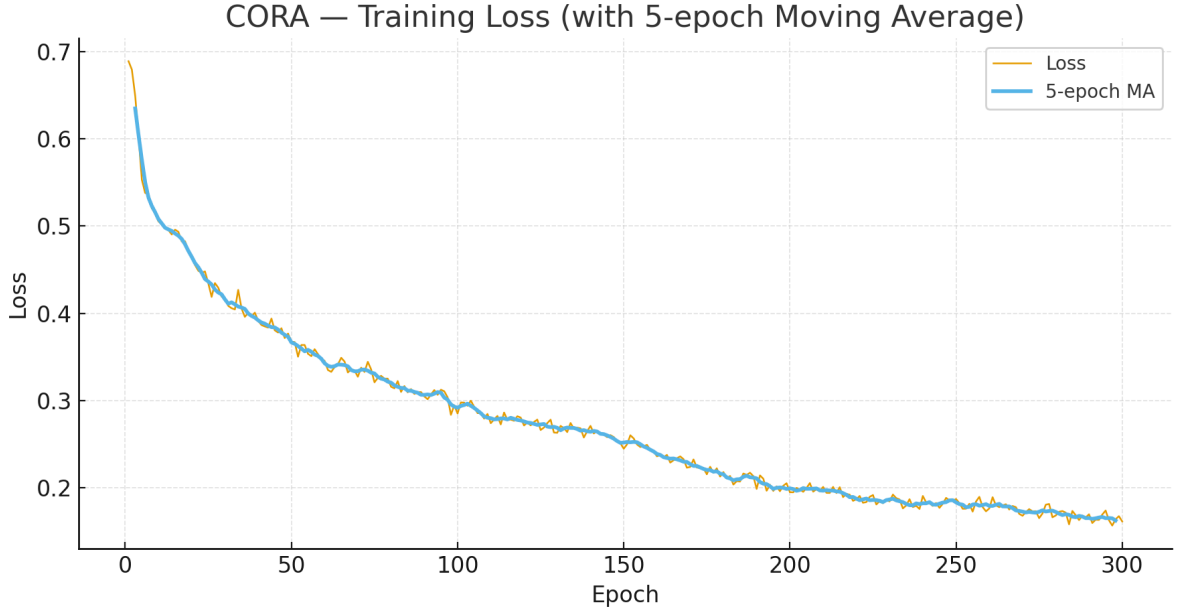


Figure 13: Loss Curve for LoRA-based Encoders with DistMult Decoders

3.4 Unified R-GAT+LoRA

This section integrates the two most effective ideas observed in earlier studies, relation-aware attention from R-GAT and parameter-efficient low-rank adaptation from LoRA, into a single encoder for link prediction. The goal is to retain the selectivity of attention over typed edges while allowing each relation to adjust its message transformation through a compact low-rank adapter. The resulting model operates within the standard encoder–decoder framework and uses the same training and evaluation settings described previously.

3.4.2 Layer formulation

Let $x_i \in R^d$ denote the embedding of node i , r_{ij} the relation type on edge $j \rightarrow i$, and $W \in R^d$ a shared projection. For each relation r LoRA introduces a low-rank pair (A_r, B_r) with $A_r, B_r \in R^{d \times k}$ and $k \ll d$, and a scalar adapter scale α . The relation-conditioned message is

$$m_{j \rightarrow i}^{(r)} = (W + \alpha A_r B_r^\top) x_j.$$

As in graph attention, attention logits are computed head-wise from the projected features of source and target:

$$\ell_{ijh} = \text{LeakyReLU}\left(\langle a^{(l)}h, z_{ih} \rangle + \langle a^{(r)}h, z_{jh} \rangle + b^{(r)}h\right)$$

$$z_{jh} = P_h m_{j \rightarrow i}^{(r)}, \quad z_{ih} = P_h x_i,$$

where $h=1, \dots, H$ indexes attention heads, $P_h \in R^{d \times c}$ are shared head projections, $a^l h, a^r h \in R^c$ are learnable attention vectors, and $b^r h$ is a relation-specific bias per head.

Normalized coefficients are

$$\alpha_{ijh} = \text{softmax}_{j \in \mathcal{N}(i)}(\ell_{ijh})$$

The head-wise aggregated representation is

$$\tilde{x}_{ih} = \sum_{j \in \mathcal{N}(i)} \alpha_{ijh} z_{jh}, \quad x_i^{\text{out}} = \begin{cases} \parallel h = 1^H \tilde{x}_{ih} & \text{(concatenate in first layer)} \\ \frac{1}{H} \sum_{h=1}^H \tilde{x}_{ih} & \text{(average in final layer).} \end{cases}$$

A two-layer stack with residual projection and layer normalization is used:

$$X^{(1)} = \text{ELU}(\text{RGAT} + \text{LoRA}(X^{(0)})), \quad Z = \text{LN}(\text{RGAT} + \text{LoRA}(X^{(1)}) + RX^{(0)}),$$

where $R^{d \times d}$ aligns the input dimension to the output.

3.4.3 Encoder–decoder scoring

Given entity embeddings Z , a triple (h, r, t) is scored by either a relation-aware DistMult decoder:

$$s_{\text{DistMult}}(h, r, t) = \langle Z_h, w_r, Z_t \rangle$$

with $w_r \in R^d$ a learnable vector per relation.

3.4.4 Learning objective and negative sampling

Training follows binary cross-entropy on mixed positive and corrupted negatives. For a batch B of positives and head/tail-corrupted negatives B^- ,

$$\mathcal{L} = -\frac{1}{|B \cup B^-|} \sum_{(h,r,t,y) \in B \cup B^-} \left[y \cdot \log \sigma(s(h, r, t)) + (1-y) \cdot \log (1 - \sigma(s(h, r, t))) \right],$$

with k negatives per positive and gradient clipping to stabilize optimization. Early stopping is performed on validation AUC, and the best checkpoint is evaluated on the test set, reporting AUC and Hits@K as in previous sections.

3.4.5 Computational considerations

To meet a VRAM 32 GB budget, the layer processes edges relation-by-relation. For each relation r , the model fetches A_r, B_r , computes $m_{j \rightarrow i}^r$, attains α_{ijh} only on edges of type r , and immediately aggregates into the output buffer. This streaming design avoids materializing tensors of shape $[E, d \times k]$ across all relations and significantly reduces peak memory without changing the forward semantics.

3.4.6 Implementation flow

Input triples are first converted into entity and relation indices, with reverse edges added to form a directed multi-graph. The encoder embeds entities and passes them through two RGAT+LoRA layers—one with head concatenation and one with head averaging—followed by residual projection and layer normalization to produce the final node embeddings. Training uses mini-batches mixing positive and corrupted triples, optimized with Adam and gradient clipping. During evaluation, AUC is computed using one-to-one negatives, and Hits@K is measured by ranking each true tail among 99 random candidates.

3.4.7 Results

Table 11 reports test performance of the unified R-GAT+LoRA encoder with DistMult across the three datasets. We defer narrative analysis to the next subsection.

Table 11: Test Results on WN18RR, FB15K-237, and Cora

Dataset	AUC	Hits@1	Hits@5	Hits@10
WN18RR	0.9559	0.6656	0.8149	0.8583
FB15K-237	0.9903	0.7142	0.9405	0.9817
Cora	0.8508	0.2884	0.5617	0.6376

3.4.8 Results Analysis

Across all datasets, the unified **R-GAT+LoRA** model consistently outperforms its GIN and GAT counterparts, confirming the complementary strengths of relation-aware attention and low-rank adaptation. On **WN18RR** and **FB15K-237**, the model achieved near-optimal AUCs of 0.9559 and 0.9903 with Hits@10 scores of 0.8583 and 0.9817 respectively, demonstrating both strong ranking capability and relational understanding. Compared to standard R-GAT, the integration of LoRA significantly boosts stability and scalability. By introducing low-rank decomposition, LoRA allows relation-specific attention to adapt dynamically without the heavy parameter cost of full matrices. This gives R-GAT+LoRA the flexibility of R-GAT’s relational attention while maintaining the efficiency of LightGCN’s linear simplicity. On the **Cora** dataset, performance gains were smaller (AUC 0.8508, Hits@10 0.6376), but the model still surpassed baseline GAT and GIN variants, showing robustness even on smaller, homophilic graphs.

The strength of R-GAT+LoRA lies in how LoRA directly addresses GAT’s key weaknesses. Traditional GATs are prone to overfitting due to over-parameterised attention heads and instability when relation data is sparse. LoRA mitigates this by constraining each relation’s transformation to a low-rank subspace, acting as a natural regulariser that improves generalisation. It also extends GAT’s fixed attention mechanism by allowing each relation to fine-tune the shared attention parameters via lightweight adapters, effectively learning *how much* and *in what direction* to adjust attention for different relation types. In short, GAT learns *what to attend to*, while LoRA teaches it *how to adapt that attention efficiently*. This synergy explains the consistent improvement of R-GAT+LoRA across datasets, making it a scalable, expressive, and memory-efficient architecture for relational graph learning.

4. Conclusion

In this report, we experimented the most effective neighbor aggregation strategy for link prediction in graphs containing diverse relation types. Our experiments across WN18RR, FB15K-237 and Cora consistently proved that relation-aware models outperform homogeneous baselines.

Among all tested architectures, the R-GAT+LoRA model proved the most effective overall. It achieved top AUC and Hits@K metrics by combining two distinct features, relation attention and parameter-efficient low-rank adaptation. The attention mechanism enables selective weighting of neighbours according to relation importance, while LoRA enhances adaptability and prevents overfitting by constraining relation transformations to a compact subspace.

The results also highlight that dataset characteristics strongly influence model performance. LightGCN excelled on sparsely-typed graphs like WN18RR and Cora, where simple structural propagation is sufficient. In contrast, GAT and GIN variants performed best on dense, diverse datasets like FB15K-237. This shows that the optimal aggregation method depends on both the model architecture and the heterogeneity of the dataset.

5. References

- Bordes, A., Usunier, N., Garcia-Durán, A., Weston, J., & Yakhnenko, O. (2013). Translating embeddings for modeling multi-relational data. *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, 2787–2795. Presented at the Lake Tahoe, Nevada. Red Hook, NY, USA: Curran Associates Inc.
- Busbridge, D., Sherburn, D., Cavallo, P., & Hammerla, N. Y. (2019). Relational Graph Attention Networks. doi:10.48550/ARXIV.1904.05811
- Chen, M., Zhang, Y., Kou, X., Li, Y., & Zhang, Y. (2021). r-GAT: Relational Graph Attention Network for Multi-Relational Graphs. doi:10.48550/ARXIV.2109.05922
- Dettmers, T., Minervini, P., Stenetorp, P., & Riedel, S. (2018). Convolutional 2D knowledge graph embeddings. *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*. New Orleans, Louisiana, USA: AAAI Press.
- GeeksforGeeks. (2025, August 4). AUC ROC Curve in machine learning. *AUC ROC Curve in Machine Learning*.
- He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 639–648. Presented at the Virtual Event, China. doi:10.1145/3397271.3401063
- Lee, G., Kim, K., & Shin, K. (2024). Revisiting LightGCN: Unexpected Inflexibility, Inconsistency, and A Remedy Towards Improved Recommendation. *Proceedings of the 18th ACM Conference on Recommender Systems*, 957–962. Presented at the Bari, Italy. doi:10.1145/3640457.3688176
- McCallum, A., Nigam, K., Rennie, J., & Seymore, K. (2000). Automating the construction of internet portals with machine learning. *Information Retrieval*, 3(2), 127–163.
- PyKEEN Project Team. (2019). Hitsatk. HitsAtK - pykeen 1.11.1 documentation.

Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., & Welling, M. (2018). Modeling Relational Data with Graph Convolutional Networks. *The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings*, 593–607. Presented at the Heraklion, Greece. doi:10.1007/978-3-319-93417-4_38

Thanapalasingam, T., Van Berkel, L., Bloem, P., & Groth, P. (2022). Relational graph convolutional networks: a closer look. *PeerJ Computer Science*, 8, e1073. doi:10.7717/peerj-cs.1073

Toutanova, K., & Chen, D. (2015). Observed versus latent features for knowledge base and text inference. In A. Allauzen, E. Grefenstette, K. M. Hermann, H. Larochelle, & S. W.-T. Yih (Eds), *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality* (pp. 57–66). doi:10.18653/v1/W15-4007

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2017). Graph Attention Networks. doi:10.48550/ARXIV.1710.10903

Vrahatis, A. G., Lazaros, K., & Kotsiantis, S. (2024). Graph Attention Networks: A Comprehensive Review of Methods and Applications. *Future Internet*, 16(9). doi:10.3390/fi16090318

Wang, K., Shen, W., Yang, Y., Quan, X., & Wang, R. (2020, July). Relational Graph Attention Network for Aspect-based Sentiment Analysis. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 3229–3238). doi:10.18653/v1/2020.acl-main.295

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2018). How Powerful are Graph Neural Networks? doi:10.48550/ARXIV.1810.00826

Zhao, W. X., Mu, S., Hou, Y., Lin, Z., Chen, Y., Pan, X., Wen, J.-R. (2020). RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. doi:10.48550/ARXIV.2011.01731

Appendix

A Performance Metrics

Area Under Curve (AUC). AUC-ROC provides insight into how effectively a classification model separates the two groups. Better model performance is shown by a higher AUC value, which implies greater ability for classification. AUC values of 1.0 and 0.5, respectively, imply perfect accuracy and random guessing (GeeksforGeeks, 2025). For the link prediction task, AUC measures the probability that a randomly chosen positive example (i.e., a true link) is ranked higher than a randomly chosen negative link.

Hits@K. Hits@K measures the proportion of true positive links that appear in the top-K ranked predictions to reflect how well the model performs in top-K retrieval scenarios (PyKEEN, 2019). Hits@K is calculated using the formula below:

$$Hits@K = \frac{\text{Number of correct predictions in top-}K}{\text{Total number of queries}}$$

Hits@1: Checks if the correct link is ranked first. Hits@5: Checks if the correct link is ranked within the top 5. Hits@10: Checks if the correct link is ranked within the top 10.